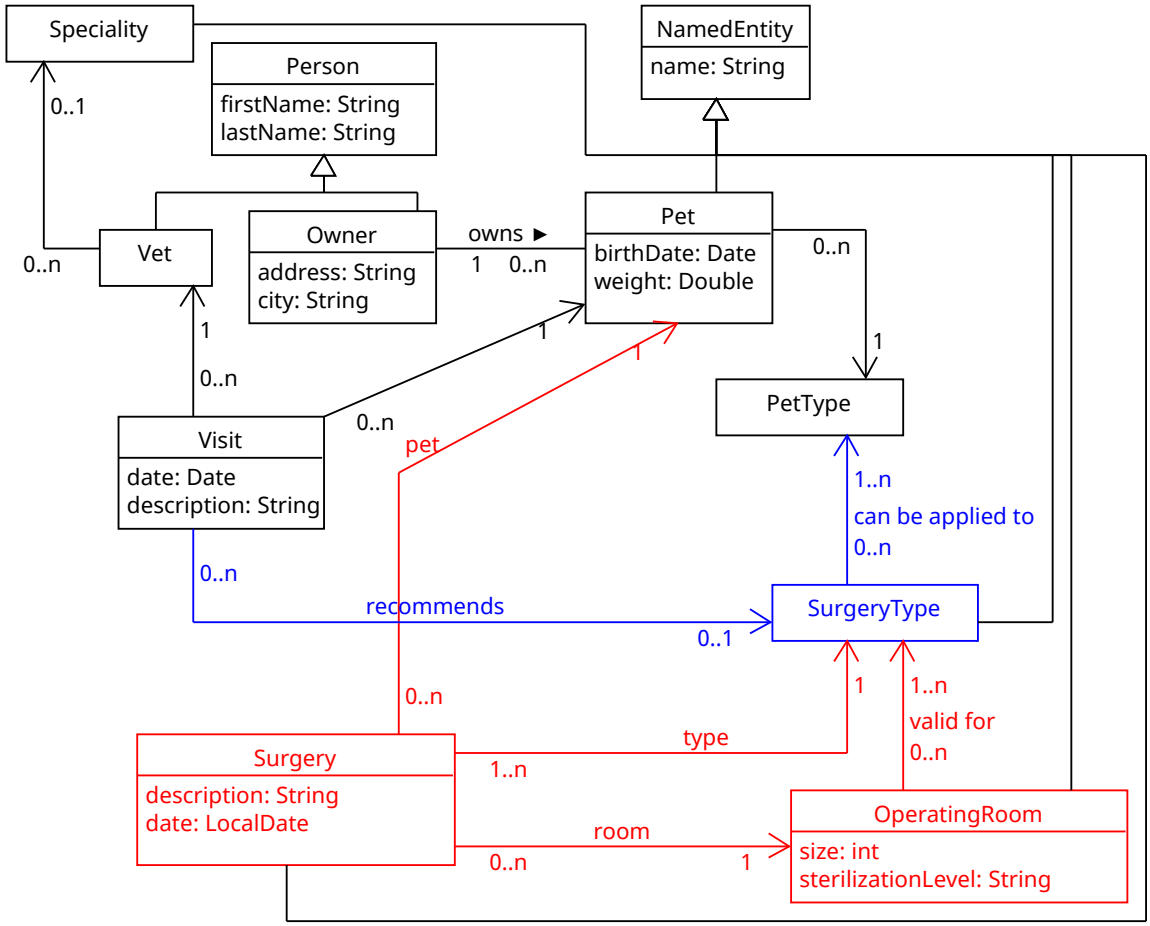
Control práctico de DP1 2024-2025 (Control-check 1)

Enunciado

En este ejercicio, añadiremos la funcionalidad de gestión de cirugías a las mascotas. En concreto, tendremos la clase “Surgery”, que representa la realización de una cirugía a una mascota por un equipo de veterinarios, y la clase “OperatingRoom”, que representa las distintas salas de operaciones disponibles en la clínica para realizar la cirugía. Cada sala de operaciones está preparada para tratar a un determinado tipo de cirugía concreto, representado con la clase “SurgeryType”. Por ejemplo, podemos tener las salas 1 y 2 preparadas para cirugía de tejidos blandos y la sala 3 preparada para cirugía oftálmica. Además, la clase SurgeryType se relaciona con aquellos PetType a los que ese tipo de cirugía puede aplicar.

El diagrama UML que describe las clases y relaciones con las que vamos a trabajar es el siguiente:



Las clases para las que realizaremos el mapeo objeto-relacional como entidades JPA se han señalado en rojo. Las clases en azul son clases que se proporcionan ya mapeadas pero con las que se trabajará durante el control de laboratorio.

Realizaremos una serie de ejercicios basados en funcionalidades que implementaremos en el sistema, y validaremos mediante pruebas unitarias. Si desea ver el resultado que arrojarían las pruebas en backend, puede ejecutarlas (bien mediante su entorno de desarrollo favorito, bien mediante el comando "*mvnw test*" en la carpeta raíz del proyecto). Cada ejercicio correctamente resuelto valdrá dos puntos, el número de casos de prueba de cada ejercicio puede variar entre uno y otro y la nota se calculará en base al porcentaje de casos de prueba que pasan. Por ejemplo, si pasan la mitad (50%) de los casos de prueba de un ejercicio, usted obtendrá un punto (2*0,5=1), y si pasan un 10% obtendrá 0,2 puntos.

Para comenzar esta prueba debe aceptar la tarea disponible en:

https://classroom.github.com/a/aLOR6cj9

Al aceptar dicha tarea, se creará un repositorio único individual para usted, debe usar dicho repositorio para realizar el control práctico. Debe entregar la actividad en EV asociada al control check proporcionando como texto la dirección url de su repositorio personal. Recuerde que además debe entregar su solución del control.

La entrega de su solución al control se realizará mediante un único comando "*git push*" a su repositorio individual. Recuerde que debe hacer `push` antes de cerrar sesión en la computadora y abandonar el aula, de lo contrario, su intento se evaluará como no presentado. Su primera tarea en este control será clonar (recuerde que si va a usar los equipos del aula para realizar el control necesitará usar un token de autenticación de GitHub como clave, tiene un documento de ayuda a la configuración en el propio repositorio del control). A continuación, deberá importar el proyecto en su entorno de desarrollo favorito y comenzar los ejercicios abajo listados. Al importar el proyecto, el mismo puede presentar errores de compilación. No se preocupe, si existen, dichos errores irán despareciendo conforme usted vaya implementando los distintos ejercicios del control.

Nota importante 1: No modifique los nombres de las clases ni la signatura (nombre, tipo de respuesta y parámetros) de los métodos proporcionados como material de base para el control. Las pruebas que se usan para la evaluación dependen de que las clases y los métodos tengan la estructura y nombres proporcionados. Si los modifica probablemente no pueda hacer que pasen las pruebas, y obtendrá una mala calificación.

Nota importante 2: No modifique las pruebas unitarias proporcionadas como parte del proyecto bajo ningún concepto. Aunque modifique las pruebas en su copia local del proyecto, éstas serán restituidas mediante un comando git previamente a la ejecución de las pruebas para la emisión de la nota final, por lo que sus modificaciones en las pruebas no serán tenidas en cuenta en ningún momento.

Nota importante 3: Mientras haya ejercicios no resueltos habrá tests que no funcionen y, por tanto, el comando `mvnw install` finalizará con error. Esto es normal debido a la forma en la que está planteado el control y no hay que preocuparse por ello. Si se quiere probar la aplicación se puede ejecutar de la forma habitual pese a que `mvnw install` finalice con error.

Nota importante 4: La descarga del material de la prueba usando git, y la entrega de su solución con git a través del repositorio GitHub creado a tal efecto forman parte de las competencias evaluadas durante el examen, por lo que no se aceptarán entregas que no hagan uso de este medio, y no se podrá solicitar ayuda a los profesores para realizar estas tareas.

Nota importante 5: No se aceptarán como soluciones válidas proyectos cuyo código fuente no compile correctamente o que provoquen fallos al arrancar la aplicación en la inicialización del contexto de Spring. Las soluciones cuyo código fuente no compile o sean incapaces de arrancar el contexto de Spring serán evaluadas con una nota de 0.

Nota importante 6: Excepto el ejercicio 5 (que depende del 4) lo ejercicios del examen son independientes y pueden ser resueltos en cualquier orden.

Test 1 – Modificar el servicio de gestión de Visitas para que no permita guardar recomendaciones de cirugía imposibles

Modificar el método *save* del servicio de gestión de visitas (*VisitService*) de manera que se lance la excepción (*UnfeasibleSurgeryException*) en caso de que se intente guardar una visita asociada a una recomendación de un tipo de cirugía que no aplica a las mascotas del tipo de la que ha venido a la visita. Por ejemplo, imaginen una visita donde el dueño trae a su pájaro y el veterinario le recomienda la cirugía de extracción de una pieza dental (que no puede ser realizada sobre ese tipo de mascotas). El método debe ser transaccional y hacer *rollback* de la transacción en caso de que se lance dicha excepción.

Test 2 – Creación de servicio y del controlador para devolver los tipos de cirugía disponibles

Parte 2A: Servicios de Gestión (1 punto):

Modificar la clase "*SurgeryTypeService*" para que sea un servicio Spring de lógica de negocio y proporcionen una implementación a los métodos que permitan:

1. Obtener todos los tipos de cirugía (*SurgeryType*) almacenados en la base de datos como una lista usando el repositorio (método *findSurgeryTypes* del servicio).
2. Obtener el tipo de cirugía (*SurgeryType)* correspondiente a un id pasado por parámetro o *null* en caso de que no exista (método *findSurgeryTypeById* del servicio).

Todos estos métodos **deben ser transaccionales**, pero las anotaciones asociadas deben realizarse a nivel de método, no a nivel de clase**.**

Parte 2B: Creación del controlador para devolver los tipos de cirugía disponibles

Crear un método en el controlador "*SurgeryTypeController"* (alojado en el paquete org.springframework.samples.petclinic.surgery) que permita devolver todos los tipos de cirugía existentes. El método debe responder a peticiones tipo GET en la URL:

http://localhost:8080/api/v1/surgerytypes

Así mismo debe crear un método para devolver un tipo de cirugía concreto, este método debe responder a peticiones en la url http://localhost:8080/api/v1/surgerytypes/X donde X es el Id del tipo de cirugía a obtener, y devolverá los datos del tipo de cirugía correspondiente. En caso de que no exista un tipo de cirugía con el id especificado el sistema debe devolver el código de estado 404 (NOT_FOUND).

Estos endpoint de la API asociados a la gestión de tipos de cirugía deberían estar accesibles únicamente para usuarios de tipo Vet (que tengan la authority "VET").

Test 3 – Creación de un componente React de listado de tipos de cirugía

Modificar el componente React proporcionado en el fichero "frontend/src/surgerytype/index.js" para que muestre un listado de las enfermedades disponibles en el sistema. Para ello debe hacer uso de la API lanzando una petición tipo GET contra la URL api/v1/surgerytypes. Un ejemplo de respuesta que podría obtener es:

```
[{
        id: 1,
        name:"Dental extractions",
```

```
        susceptiblePetTypes:[{name: "dog", id:2}, {name: "cat", id:1}, {name: "snake", id:4}]
},
{
        id:2,
        name:"Bladder surgery",
        susceptiblePetTypes:[{ name: "cat", id:1}, {name: "hamster", id:6}]
}]
```

Este componente debe mostrar los tipos de cirugía en una tabla (se recomienda usar el componente *Table* de *reactstrap*) incluyendo columnas para su nombre, y el conjunto de tipos de mascotas al que aplica este tipo de cirugía. Para esto último, el componente debe mostrar en la celda asociada una lista (preferiblemente no ordenada <ul>) con el nombre de los tipos de mascotas a los que aplica en la celda correspondiente.

Para poder lanzar este test y comprobar su resultado puede colocarse en la carpeta de *frontend* y ejecutar el comando `npm test` y pulsar 'a' en el menú de comandos de jest. Nótese que previamente debe haber lanzado al menos una vez el comando `npm install` para que todas las librerías de node estén instaladas.

Test 4 – Creación de las entidades Surgery y OperatingRoom y sus repositorios asociados

Modificar las clases "*Surgery*" y "*OperatingRoom*" para que sean entidades y eliminar las anotaciones @Transient de estas clases. Estas clases están alojadas en el paquete "org.springframework.samples.petclinic.surgery", y deben tener los siguientes atributos y restricciones:

Para ambas clases:

- El atributo de tipo entero (Integer) llamado "*id*" actuará como clave primaria en la tabla de la base de datos relacional asociada a la entidad.
- Un atributo de tipo cadena de caracteres (String) llamado "*name*" obligatorio (no puede ser nulo), que debe tener una longitud mínima de 3 caracteres y máxima de 50 y que no puede estar formada por caracteres vacíos (espacios, tabuladores, etc.).

Para la clase *Surgery*:

- Un atributo de tipo fecha (LocalDate) llamado "*date*", que representa la fecha en que se realiza la cirugía. Seguirá el formato "dd/MM/yyyy" (puede usar como ejemplo la clase *Pet* y su fecha de nacimiento para ver cómo se especificar dicho formato, pero nótese que el patrón del formato es distinto). Este atributo debe ser obligatorio y se almacenará en la BD con el nombre de columna "surgery_date".
- El atributo de tipo cadena caracteres (String) llamado "*description*" opcional.

Para la clase *OperatingRoom*:

- Un atributo de tipo entero llamado "*size*" obligatorio, que representa el tamaño de la sala en metros cuadrados, y tiene que ser un número mayor que cero.
- Un atributo de tipo cadena de caracteres (String) llamado "*sterilizationLevel*" obligatorio que únicamente podrá tomar tres valores: "FULL", "SEMI", "LOW"[1].

[1] La expression regular para implementar dicha restricción es "^(FULL)|(SEMI)|(LOW)$"

Modificar las interfaces "*SurgeryRepository*" y "*OperationRoomRepository*" alojadas en el mismo paquete para que extiendan a *CrudRepository*. No olvide especificar sus parámetros de tipo.

Se pide crear las siguientes relaciones entre las entidades:

- Una relación unidireccional desde "*Surgery*" hacia "*Pet*" que exprese la que aparece en el diagrama UML (mostrado en la primera página de este enunciado) respetando sus cardinalidades, usando el atributo "*pet*".
- Una relación unidireccional desde "*Surgery*" hacia "*SurgeryType*" que represente la que aparece en el diagrama UML respetando sus cardinalidades, usando el atributo "*type*".
- Una relación unidireccional desde "*OperatingRoom*" hacia "*SurgeryType*" que represente la que aparece en el diagrama UML respetando sus cardinalidades, usando el atributo "validFor".
- Una relación unidireccional desde "*Surgery*" hacia "*OperatingRoom*" que exprese la que aparece en el diagrama UML respetando sus cardinalidades, usando el atributo "*room*" de la clase "*Surgery*".

Test 5 – Modificación del script de inicialización de la base de datos para incluir dos cirugías y dos salas de operaciones

Modificar el script de inicialización de la base de datos, para que se creen las siguientes cirugías (*Surgery*) y salas de operaciones (*OperationRoom*):

Surgery 1:

- id: 1
- name: "Fang extraction"
- description: null
- date: 16/10/2024
- room: OperatingRoom con id=1
- pet: Pet con id = 3
- type: SurgeryType con id = 1

Surgery 2:

- id: 2
- name: "Removal of a bladder stone"
- description: "Removal of a bladder stone to a 10-year-old cat."
- date: 17/10/2024
- room: OperatingRoom con id=2
- pet: Pet con id=1
- type: SurgeryType con id=2

OperatingRoom 1:

- id: 1
- name: “Room for dental surgery”
- size: 15
- sterilizationLevel: “SEMI”
- Asociado con los SurgeryType con ids 1, 3 y 4

OperatingRoom 2:

- id: 2
- name: “Room for soft tissue surgery”
- size: 20
- sterilizationLevel: “FULL”
- Asociado con los SurgeryType con ids 2 y 5

Tenga en cuenta que el orden en que aparecen los INSERT en el script de inicialización de la base de datos es relevante al definir las asociaciones.

CONTROL CHECK 1 - GRUPO Ing - 24/25

TEST 1

```
@Transactional(rollbackFor = UnfeasibleSurgeryException.class)
public Visit save(Visit visit) throws DataAccessException,
UnfeasibleSurgeryException {

   if (visit.getRecommends() != null &&

!visit.getRecommends().getSusceptiblePetTypes().contains(visit
.getPet().getType())) throw new UnfeasibleSurgeryException();
   else return visitRepository.save(visit);
}
```

TEST 2 A

```
@Service
public class SurgeryTypeService {
   SurgeryTypeRepository repo;

   public SurgeryTypeService(SurgeryTypeRepository repo) {
       this.repo = repo;
   }

   @Transactional(readOnly = true)
   public SurgeryType findSurgeryTypeById(Integer i) {
       Optional<SurgeryType> st = repo.findById(i);
       return st.isPresent()? st.get():null ;
   }


   @Transactional(readOnly=true)
   public List<SurgeryType> findSurgeryTypes() {
       return repo.findAll();
   }

}
```

TEST 2B

```
@RestController
@RequestMapping("/api/v1/surgerytypes")
public class SurgeryTypeController {

   SurgeryTypeService surgeryTypeService;
```

```
    @Autowired
    public SurgeryTypeController(SurgeryTypeService
surgeryTypeService) {
    this.surgeryTypeService = surgeryTypeService;}

    @GetMapping
    @ResponseStatus(HttpStatus.OK)
    public ResponseEntity<List<SurgeryType>> getAll() {
        return new
ResponseEntity<>(surgeryTypeService.findSurgeryTypes(),
HttpStatus.OK);
    }
    @GetMapping("/{id}")
    @ResponseStatus(HttpStatus.OK)
    public ResponseEntity<SurgeryType>
getSurgeryType(@PathVariable("id") Integer id){
        SurgeryType st =
surgeryTypeService.findSurgeryTypeById(id);
        if(st==null){
            throw new ResourceNotFoundException("Product not
found");
        }
          return new ResponseEntity<>(st, HttpStatus.OK);
    }
}
```

SecurityConfiguration.java

```
.requestMatchers("/api/v1/surgerytypes/**").hasAuthority("VET"
)
```

TEST 4

```
@Getter
@Setter
@Entity
public class Surgery extends NamedEntity {

    String description;

    @Column(name = "surgery_date")
    @DateTimeFormat(pattern = "dd/MM/yyyy")
    @NotNull
    private LocalDate date;

    @ManyToOne(optional = false)
```

```
    SurgeryType type;

    @ManyToOne(optional=false)
    Pet pet;

    @ManyToOne(optional=false)
    OperatingRoom room;
}
```

```
@Getter
@Setter
@Entity
public class OperatingRoom extends NamedEntity {

    @NotNull
    @Min(1)
    int size;

    @Pattern(regexp = "^(FULL)|(SEMI)|(LOW)$")
    String sterilizationLevel;

    @ManyToMany
    @NotNull
    Set<SurgeryType> validFor;
}
```

```
public interface SurgeryRepository extends
CrudRepository<Surgery, Integer>{
    Optional<Surgery> findById(Integer i);

    List<Surgery> findAll();

    Surgery save(Surgery any);

}
```

```
public interface OperatingRoomRepository extends
CrudRepository<OperatingRoom, Integer>{

    Optional<OperatingRoom> findById(Integer i);

    List<OperatingRoom> findAll();
```

```
    OperatingRoom save(OperatingRoom any);

}
```

TEST 5

```
INSERT INTO operating_room(id, name, size, sterilization_level) VALUES (1,
'Room for dental surgery', 15, 'SEMI');
INSERT INTO operating_room(id, name, size, sterilization_level) VALUES (2,
'Room for soft tissue surgery', 20, 'FULL');

INSERT INTO surgery(id, name, description, surgery_date, room_id, pet_id,
type_id) VALUES (1, 'Fang extraction', null,'2024-10-16', 1, 3, 1);
INSERT INTO surgery(id, name, description, surgery_date, room_id, pet_id,
type_id) VALUES (2, 'Removal of a bladder stone', 'Removal of a bladder
stone to a 10-year-old cat.','2024-10-17', 2, 1, 2);

INSERT INTO operating_room_valid_for(operating_room_id, valid_for_id)
VALUES (1, 1), (1, 3), (1, 4), (2, 2), (2, 5);
```